

Module 2 Project

This project runs throughout the course and involves a GitHub repository that contains the code for a Python package called `risk_utils`.

By the end of Day 3, your team should be able to clone this repo, make changes to it, then submit a request for the administrator of the repository to pull these changes to the main branch.

Don't worry if some of the terms above are not clear to you at this point; the project is broken down into sections that match the pace of the course material.

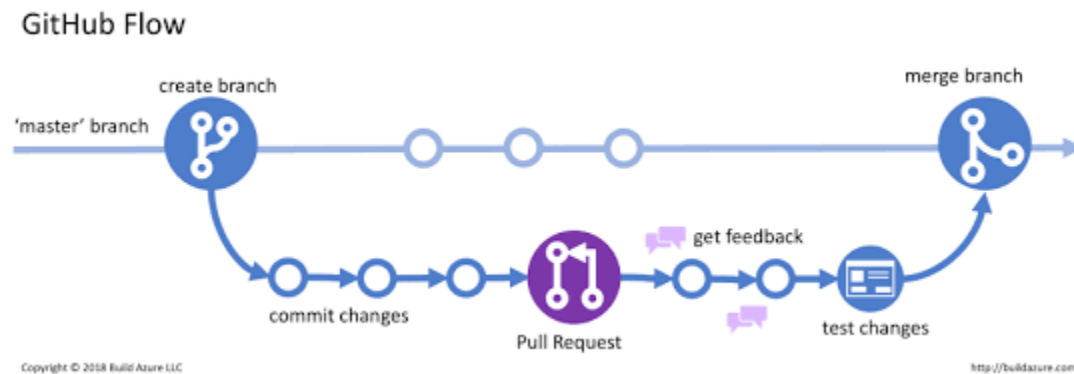
Task 1: Loading files into a local database

Use what you learned at the end of the day to modify the first 2 functions in the file (**FinalPackage/risk_utils/external_io.py**) to read all the files in the data folder into a DuckDB database. Run the file (**test_io.py**) in the same folder to make sure this was done correctly.

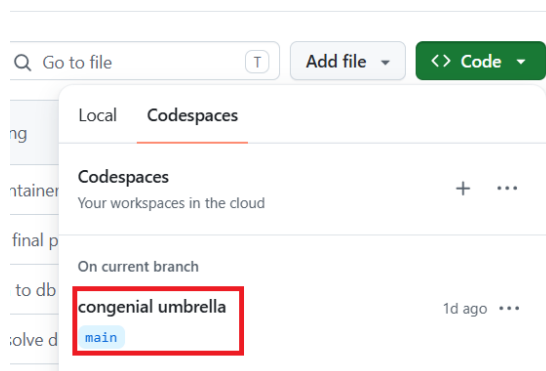
How many rows were loaded into the database?

Task 2: The GitHub PR workflow

Having learned the concepts of cloning, branching, and pull requests, it is now time to put this into action:

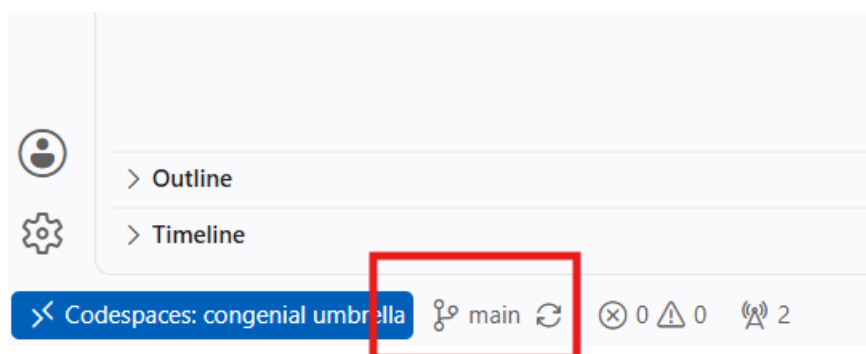


1. **Open Codespace:** Click the green **Code** button on GitHub and open your Codespace



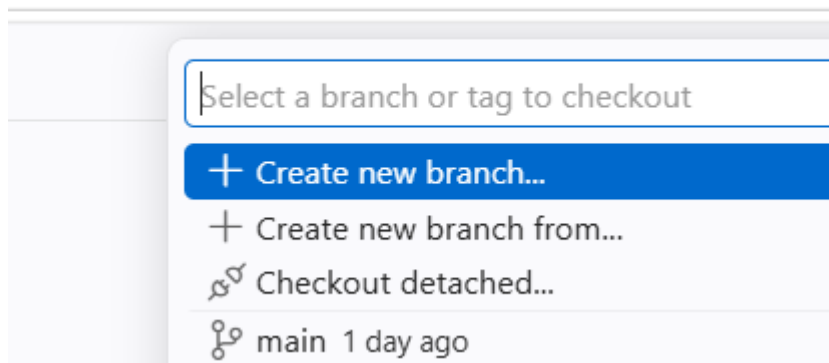
2. **Create Your Branch:**

- Click the current branch indicator on the **bottom-left status bar** (usually says main)



- Select **"Create new branch..."** from the top command menu.

:hub.dev

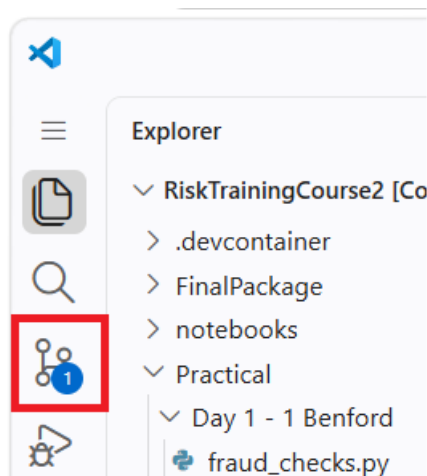


- Type your branch name using your group and name (e.g., group1-faisal) and press **Enter**.

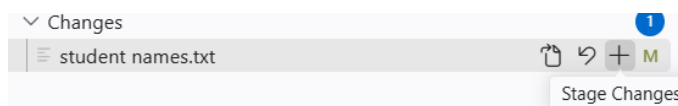
3. **Edit the File:** Open Student Names.txt from the Explorer panel and add your name on a new line.

4. **Stage & Commit:**

- Open the **Source Control** tab on the left sidebar (icon with three connected dots).

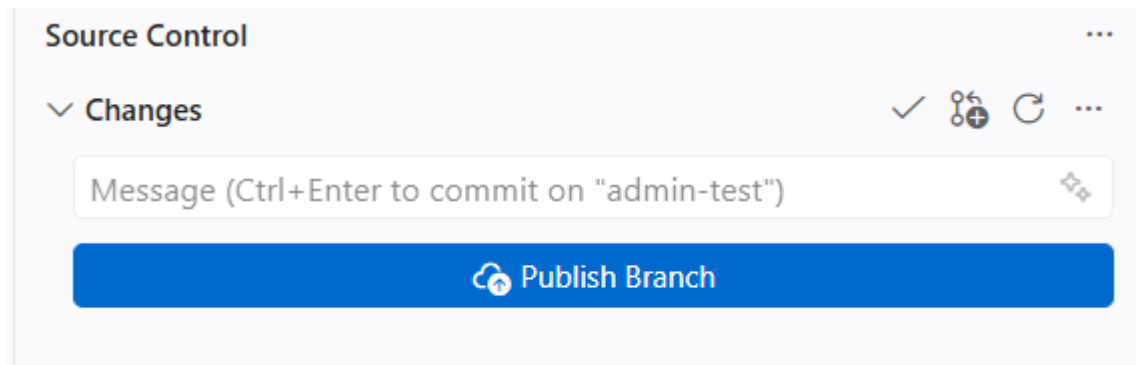


- Hover over Student Names.txt under **Changes** and click the + (**Stage Changes**) icon.

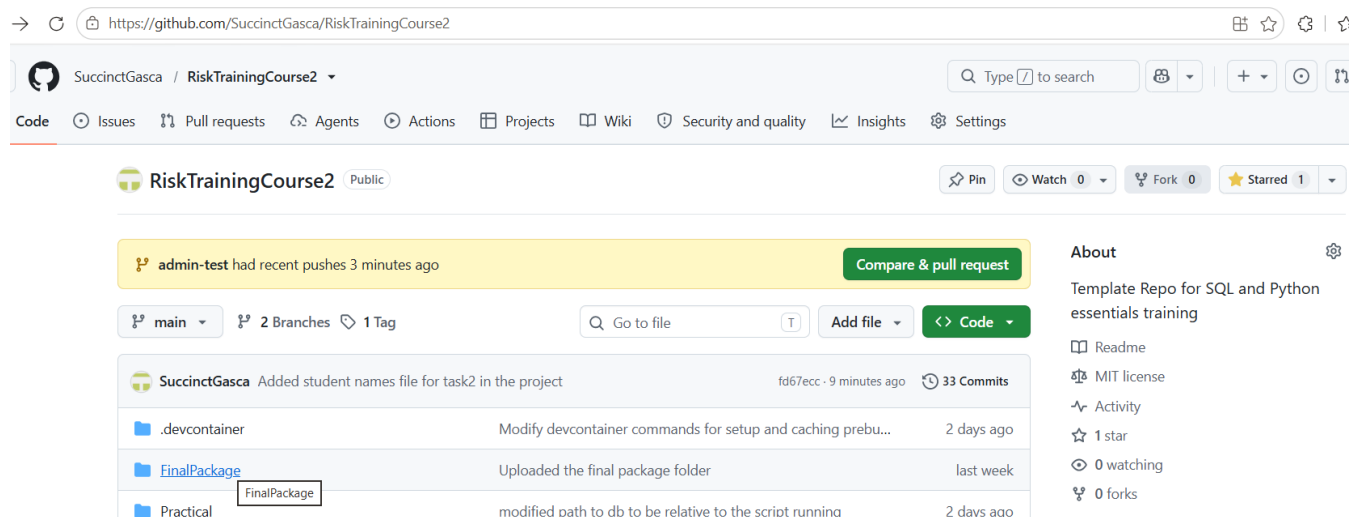


- Type "Add YOUR NAME to Student Names" into the message box at the top – use your actual name!
- Click the blue **Commit** button.

5. **Publish Branch:** Click the blue **Publish Branch** button that appears in the Source Control panel (or the cloud icon on the bottom-left status bar).



6. **Initiate Pull Request:** Navigate back to the repository page on GitHub, click **Compare & pull request**, and submit. Your admin will be notified of your changes and can review them before accepting them on the main branch!



Conceptual Background: OOP vs. SQL in Risk Analytics

When building risk analytics systems, choosing where to place detection logic depends on scope and data locality:

- **Python OOP Methods (models.py):** Ideal for single-record logic. Methods operate on individual transaction objects using instance attributes (self.amount, self.payment_format). Use OOP when validating inputs, converting values, or applying business rules on single events.
- **SQL / DuckDB Queries (fraud.py):** Ideal for set-based logic. Queries run directly against database tables (con connection) to evaluate thousands of rows at once.

Use SQL when computing aggregations (GROUP BY), finding circular links (JOIN), or tracking time-series patterns.

Task 3: Single-Record Detection with OOP Inheritance (models.py)

Scenario: You need to extend the base Transaction class in risk_utils/models.py to create a specialized AMLTransaction class that handles advanced single-transaction checks.

Instructions:

Create a branch named after your group name e.g. "Group-1" like how you created a branch in Task 2.

ONLY 1 person in the group will commit to this branch. Choose this one person but collaborate on the code.

Create an AMLTransaction class inside risk_utils/models.py that inherits from Transaction, then implement the following three methods:

1. Payment Format Anomaly (is_payment_format_anomaly)

AML Rule: High-value transfers settled via non-standard channels (like reinvestments) often bypass standard retail payment controls.

- *Requirement:* Return True if is_large() is True AND payment_format == 'Reinvestment'.

2. Cross-Bank Transfer Check (is_cross_bank_transfer)

- *AML Rule:* Moving funds between different banking institutions is a common technique used during the layering phase to fragment audit trails.
- *Requirement:* Return True if from_bank != to_bank.

3. Currency FX Spread Loss (is_fx_spread_anomaly)

- *AML Rule:* Money launderers often accept poor exchange rates or deliberate financial losses just to convert illicit funds into a clean currency.
- *Requirement:* Convert both Amount_Paid and Amount_Received to USD using self._converter.toUSD(). Return True if receiving_currency != payment_currency AND the converted received value reflects a loss $\geq 5\%$ compared to the converted paid value.

Task 4: Set-Based Analytics with DuckDB SQL (fraud.py)

Scenario: Single-record checks cannot detect multi-account networks or fund velocity. You will write set-based SQL functions in risk_utils/fraud.py that query the DuckDB database.

Instructions:

Continue to work on your group branch from Task 3.

Add the following three SQL functions to risk_utils/fraud.py (apply the @timer decorator to each):

1. **Scatter Payments / Fan-Out Rule** (flag_scatter_payments(con, max_recipients=5))
 - *AML Rule:* A single account sending rapid payments to numerous distinct recipients is a classic signature of fan-out layering.
 - **Requirement:**

Write a SQL query grouping by From_Account that returns accounts sending transfers to ≥ 5 unique To_Account destinations.
2. **Round-Trip / U-Turn Rule** (flag_round_trips(con))
 - *AML Rule:* Circular transfers between two parties (Account A $\rightarrow$ Account B $\rightarrow$ Account A) are used to manufacture artificial transactional volume or wash funds.
 - **Requirement:**

Write a SQL JOIN query matching t1.From_Account = t2.To_Account AND t1.To_Account = t2.From_Account to return circular transfer pairs.
3. **Rapid Pass-Through Shell Rule** (flag_rapid_pass_through(con, threshold_ratio=0.9))
 - *AML Rule:* Shell accounts typically hold zero net balance, instantly transferring out almost all incoming funds to the next layer.
 - **Requirement:**
 - Write a CTE query calculating total outbound SUM(Amount_Paid) vs. inbound SUM(Amount_Received) per account, returning accounts where outbound flow is $\geq 90\%$ of inbound flow.